

Neuromorphic Spiking Attention Plugin for Local AI Clients: An Apple Silicon Optimized MCP Architecture

Neuromorphic Foundations of Spiking Attention

Traditional artificial neural networks, including large language models based on the Transformer architecture, utilize dot-product similarity mechanisms to calculate attention maps across dense matrices of floating-point numbers.¹ While highly expressive, this mathematical approach scales quadratically with sequence length, creating a significant memory bottleneck during inference.¹ This bottleneck is aggravated by the von Neumann architectural constraint, which requires continuous transfer of token projection matrices between main memory and processing units.² To overcome these constraints, modern cognitive engineering looks to biological neural systems, where attention emerges naturally from discrete, event-driven spike communication across highly sparse networks.¹ Biological intelligence relies on temporal coordination rather than continuous matrix multiplications, utilizing localized learning rules to encode complex cognitive relationships.¹

The transition to a neuromorphic paradigm is facilitated by replacing standard dot-product calculations with Spiking Spike-Timing-Dependent Plasticity (STDP) Transformer dynamics (S^2TDPT).¹ Rather than storing and processing dense attention matrices, correlation patterns are embedded directly within synaptic weights through temporal alignment.¹ The core mathematical framework is driven by discrete-time representations of Leaky Integrate-and-Fire (LIF) neuron dynamics⁵:

$$U[t + 1] = \beta \cdot U[t] + X[t + 1] - S[t] \cdot V_{thr}$$

where U is the membrane potential, X is the input synaptic current, $S \in \{0, 1\}$ represents the binary output spike event, $\beta \in (0, 1)$ is the temporal decay factor, and V_{thr} is the constant spike threshold.⁵

To bypass the energy-intensive exponential and layer normalization functions characteristic of deep artificial models, the neuromorphic substrate integrates a Power-of-Two softmax formulation (PTsoftmax) alongside a Bit Shifting PowerNorm (BSPN).⁷ By restricting calculations to base-two exponential values, exponential operations are replaced with hardware-friendly bit-shifting operations.⁷ This simplification reduces synaptic operations, decreases the system's memory footprint, and results in substantial energy savings during real-time on-device inference.¹

Information salience is encoded in precise spike timing rather than magnitude.¹ Temporal correlations between pre-synaptic and post-synaptic events dictate synaptic plasticity.¹ This

behavior is modeled via localized STDP rules⁸:

$$\Delta w = \begin{cases} A_+ \exp\left(-\frac{\Delta t}{\tau_+}\right) & \text{if } \Delta t > 0 \\ -A_- \exp\left(\frac{\Delta t}{\tau_-}\right) & \text{if } \Delta t \leq 0 \end{cases}$$

where $\Delta t = t_{\text{post}} - t_{\text{pre}}$ represents the precise arrival difference between pre-synaptic and post-synaptic spikes, A_+ and A_- govern the maximum scales of potentiation and depression, and τ_+ and τ_- act as asymmetric temporal time constants.⁸ By using these biologically grounded operations, the system achieves a multiplication-free, addition-only formulation.¹ The heavy, energy-intensive softmax operations are completely bypassed, eliminating the requirement to explicitly materialize intermediate attention score matrices in memory.¹ To interface continuous vector embeddings generated by modern language models with discrete spiking networks, the architecture implements a dimension-independent z-score standardized top- k population code.⁸ This encoding maps dense coordinate dimensions into active firing patterns with high discrimination retention across varying dimensionality constraints.⁸ An embedding vector $E \in \mathbb{R}^d$ is first standardized to derive coordinate-wise z-scores:

$$Z_i = \frac{E_i - \mu_E}{\sigma_E}$$

Neurons corresponding to coordinate indices i that fall within the top- k percentile of the distribution are triggered to fire a localized spike ($S_i = 1$), whereas the remaining populations remain silent ($S_i = 0$).⁸ This standardized projection decouples the physical dimensionality of the upstream model from the downstream spiking substrate, maintaining stable associative recall dynamics without requiring explicit recalibration for different model variants.⁸

Unified Memory and Hardware-Level Optimization on Apple Silicon

Executing complex spiking simulations on consumer hardware typically introduces severe thermal and processing overhead, particularly when standard machine learning libraries are forced to bridge data across discrete CPU and GPU boundaries.⁵ On Apple Silicon, this bottleneck is resolved by utilizing the MLX framework, a specialized numerical computation engine designed from the ground up for unified memory architectures.⁵ By leveraging unified memory, the physical RAM of the host Mac device is shared directly between the CPU and the M-series GPU.⁵ There is no requirement to copy large tensor projections across PCI buses, enabling the local stack to scale temporal windows and manage multi-layered spiking structures with minimal resource impact.⁵

The backend of the spiking engine is constructed natively with the mlx-snn library, which

inherits directly from `mlx.core`.⁶ The library uses functional state passing, where the active state of every simulated neuron is managed as a standard Python dictionary rather than a mutable, hidden class state.⁵ This explicit state pattern is compatible with MLX's high-level functional transforms⁵:

- **Lazy Evaluation (`mx.eval`):** Operations build a deferred computation graph that executes only when a result is explicitly demanded.⁵ For temporal spiking loops, the entire unrolled execution graph across T timesteps can be optimized, scheduled, and compiled as a unified operation, reducing memory allocation cycles.⁵
- **Just-In-Time Compilation (`mx.compile`):** The state transitions of recurrent spiking layers are JIT-compiled into optimized Metal kernels, executing natively on the M-series GPU accelerators.⁶
- **Immutable Array Updates:** MLX prohibits in-place array mutations to ensure mathematical purity in the computation graph.⁵ Neuron potential resets must be declared as functional assignments (e.g., `mem = beta * mem + x` rather than `mem += x`), allowing the compiler to optimize the memory hierarchy.⁵

The system incorporates a multi-scale leaky integrate-and-fire formulation (MSLeaky) to process multi-scale temporal context.¹¹ Rather than utilizing a uniform decay factor, MSLeaky assigns frequency-matched decay rates (β_j) to parallel spiking branches.¹¹ This layout mimics biological delta, theta, alpha, beta, and gamma oscillations, capturing both immediate transient changes and long-term contextual patterns in a single network.¹¹ To scale temporal sequences without running out of RAM, the training loop implements chunked backpropagation-through-time (`chunked_bppt_forward`) and state detachment (`detach_state`), which isolates the computation graph at defined temporal boundaries.¹¹ Gradient propagation through the non-differentiable Heaviside step function is managed using a custom Straight-Through Estimator (STE) pattern.⁵ This pattern works around current MLX limitations with vector-Jacobian products (VJPs) by blending a smooth surrogate gradient with the discrete forward thresholding operation:

$$\text{Output} = \text{stop_gradient}(\Theta(x) - \tilde{\sigma}(x)) + \tilde{\sigma}(x)$$

where $\Theta(x)$ represents the unit step function, and $\tilde{\sigma}(x)$ is a differentiable surrogate function like the fast sigmoid or arctangent approximation.⁵

Neuron Model	Update Equations	State Vector	Dynamics & Presets
Leaky Integrate-and-Fire (LIF)	$U[t + 1] = \beta U[t] + X[t]$ ⁵	<code>state["mem"]</code> ¹¹	Standard first-order decay ⁵

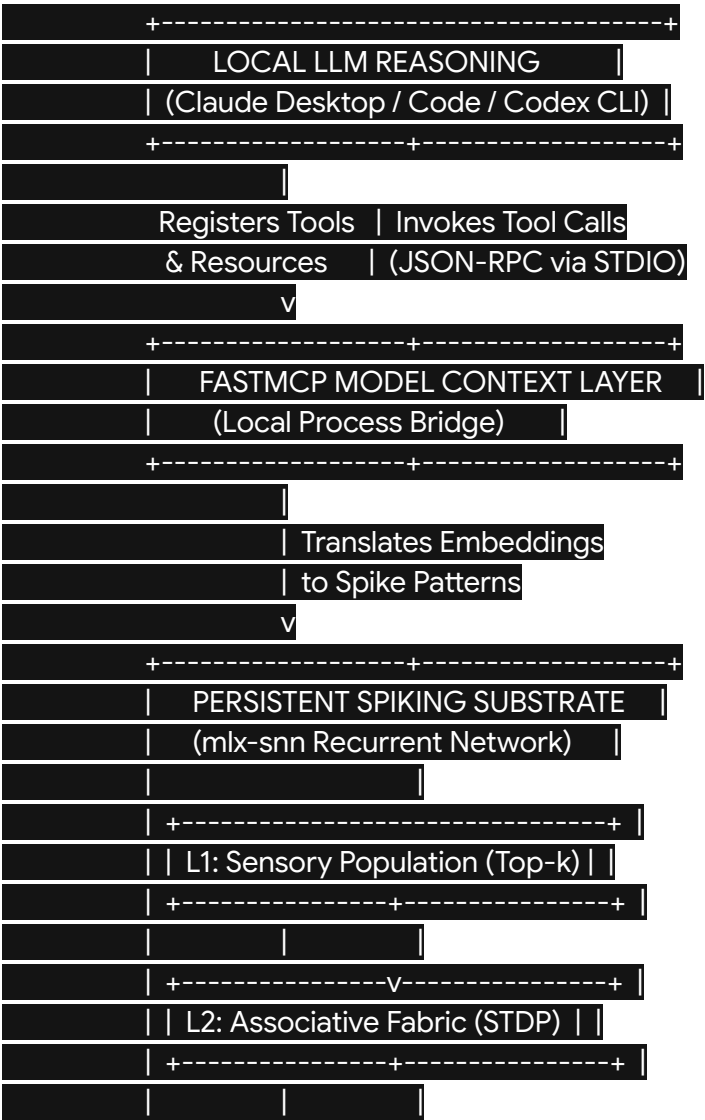
Integrate-and-Fire (IF)	$U[t + 1] = U[t] + X[t]$ ⁵	state["mem"] ¹¹	Non-leaky perfect integration ($\beta = \infty$) ⁵
Adaptive LIF (ALIF)	$U[t + 1] = \beta U[t] + X[t]$; $A[t + 1] = \rho A[t] +$ ⁵	state["mem"], state["adapt"] ¹¹	Spike-frequency adaptation with dynamic threshold ⁵
Multi-Scale Leaky (MSLeaky)	Branch-specific β_j decay rates	state["mem"] per branch ¹¹	Frequency-matched decay modeling EEG bands ¹¹
Recurrent LIF (RLeaky)	Integrates learnable recurrent synaptic feedback	state["mem"], state["spk"] ¹¹	Recurrent layer topologies with local loops ¹¹

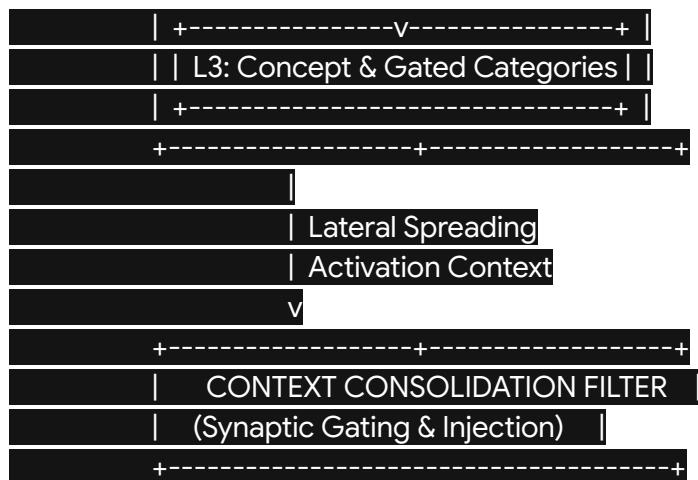
Computational Backend	Peak GPU Allocation	Sequence Scaling Limit	Compilation Compatibility	Efficiency Factor (relative to V100/MPS)
mlx-snn (Apple M-Series Native)	61 MB – ⁵	> 10 ⁶ unrolled steps	mx.compile (Full JIT graph execution) ⁵	1.0× baseline, 17–25× system efficiency ¹¹
snnTorch (Apple Silicon MPS)	241 MB – ⁵	10 ³ unrolled steps (VRAM wall)	Limited tracing, VJP shape mismatches ⁵	2.0–2.5× slower execution latency ⁵
snnTorch (Host CPU Execution)	System RAM bound	System memory execution limits	Standard TorchScript JIT	3.0–10.0× slower execution latency ⁶

Hybrid Cognitive Architecture and Spreading

Activation

The implementation decouples abstract semantic operations from associative memory consolidation.⁸ Traditional retrieval-augmented generation (RAG) isolates historical data within static databases, retrieving information via mathematical vector similarity searches that fail to capture historical co-activation structures.⁸ The hybrid SNN-LLM architecture, inspired by the EMBER framework, places the localized LLM inside a persistent, biologically grounded memory substrate.⁸





The system is organized into a multi-tiered spiking neural network featuring hierarchical sensory, concept, and categorical layer topologies.⁸ Layer 1 acts as a fixed sensory translation barrier, mapping standardized input embeddings into precise spatial spike distributions.⁸ Layer 2 functions as the plastic associative fabric, where dynamic connections are established, potentiated, or depressed based on temporal co-occurrence via active STDP rules.⁸ Layer 3 maps these associations into abstract conceptual groupings, stabilizing memory recall.⁸ Rather than executing a database query, the system streams prompt embeddings directly into the sensory layer as localized spike patterns.⁸ This stimulus propagates laterally through the recurrent connections of the associative fabric, triggering spreading activation.⁸ This lateral cascade acts as an implicit, neuromorphic attention filter, selectively amplifying historical concepts that are dynamically related to the current context.⁸ Because the network is configured with balanced excitatory and inhibitory populations, self-limiting lateral inhibition prevents runaway activation cascades, ensuring that only the most relevant, tightly coupled memory pathways fire.⁸

To structuralize this stream, the architecture integrates a dual-graph memory protocol called Hebbian Distillation (derived from HeLa-Mem) alongside a Bayesian Surprise Event Segmenter (derived from EM-LLM).¹⁴ Incoming text sequences are analyzed by the Bayesian surprise engine, which monitors semantic transitions and projects graph-theoretic boundaries around distinct events in real-time.¹⁸ These segmented events are registered as co-activation vectors within the episodic memory graph.¹⁴ As connections strengthen through repeated co-activation, a background distillation process detects densely connected memory hubs and refines them into structured semantic knowledge, mirroring the episodic-semantic distinction in human cognition.¹⁴

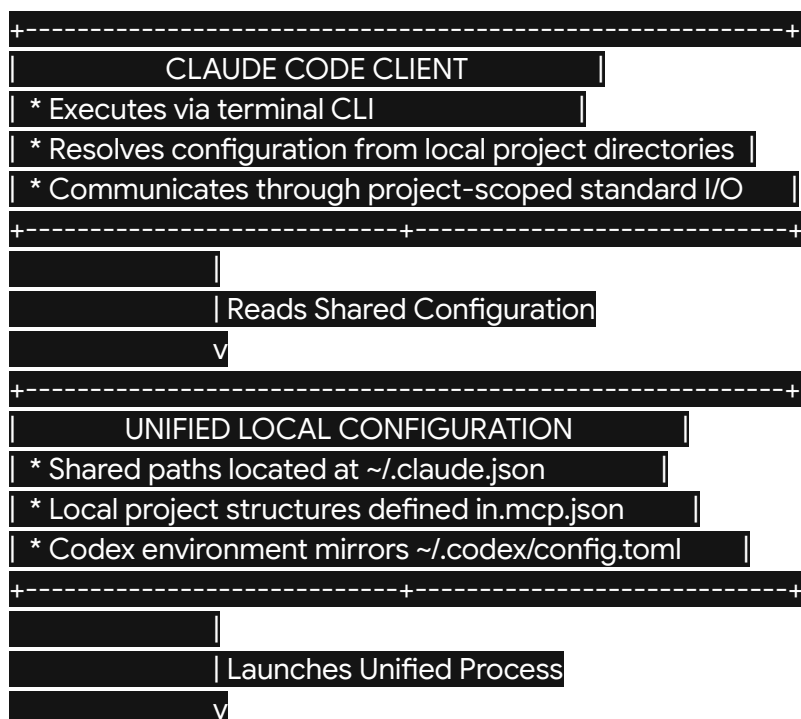
Synaptic connections are dynamically gated by contextual configurations, mimicking the prefrontal cortex context-gating mechanisms.¹⁹ An external context signal, derived from active runtime parameters or historical sequences, selectively silences specific neural clusters, reconfiguring the effective network connectivity map.²⁰ This architectural design enables local task switching with zero mutual interference, preserving consolidated memory spaces from

catastrophic forgetting during multi-turn local interactions.¹⁹

Rapid Implementation Blueprint and Configuration Shortcuts

The local plugin integrates with Codex Client, Claude Desktop, and Claude Code through the Model Context Protocol (MCP).²¹ The MCP acts as a standard middleware layer, enabling local LLM clients to communicate with background processes via JSON-RPC 2.0 messages.¹⁵ To minimize runtime overhead and eliminate the complexity of network handshakes, the transport layer is established entirely over standard input/output (stdio) channels.¹⁵ Standard output (stdout) is reserved strictly for JSON-RPC serialization, while standard error (stderr) is utilized for logging, preventing packet corruption.²⁵

The primary rapid-development shortcut lies in configuring a single, unified local FastMCP python process communicating over stdio.¹⁵ This architecture allows Claude Code, Codex Client, and Claude Desktop to point to the exact same local python runtime.²⁶ By accessing a single background process, these different interfaces share the cached neural weights, connection mappings, and local state.²⁶ This design provides a unified, cross-application memory system without requiring complex sync daemons or file lock managers.²⁷



```
+-----+
| FASTMCP NEUROMORPHIC DAEMON |
| * Listens on stdio using JSON-RPC protocol |
| * Installs to Claude Desktop via fastmcp CLI |
| * Executes on native Apple Silicon GPU via Metal |
+-----+
```

To accelerate installation, developers can use the FastMCP CLI to automatically register the server with the local Claude Desktop configuration using a single terminal command.³¹ The CLI scans active local editor instances and configures environment paths, eliminating manual editing of deep system directories.³² Additionally, when spawning the MCP server, Claude Code injects the CLAUDE_PROJECT_DIR environment variable.²⁹ The spiking engine reads this parameter to locate workspace-specific weights and adapt its context gating dynamically.²⁹

Local Client	Configuration Path	Scope Precedence	Core Transport
Claude Code	~/.claude.json ²⁸	Local > Project > User ²³	stdio process command ²³
Claude Desktop	~/Library/Application Support/Claude/claude_desktop_config.json ²⁵	Global User Environment	stdio process command ²⁵
Codex Client	~/.codex/config.toml ²²	User / Project-scoped .codex/config.toml ²⁶	stdio table process mapping ²⁶

Complete System Wireframes and Implementation Code

The implementation of the spiking attention plugin consists of three primary code layers: the FastMCP server wrapper, the mlx-snn simulation backend, and the client configuration manifests.

FastMCP Server Wrapper (mcp_server.py)

This file implements the MCP server interface, exposing high-level tools for neuromorphic attention encoding and offline consolidation.

```

Python
import OS
import sys
import logging
from typing import Dict, Any, List
from fastmcp import FastMCP
import mlx.core as mx

# Ensure all standard logging is directed safely to stderr to avoid corrupting stdio transport
logging.basicConfig(level=logging.INFO, stream=sys.stderr)
logger = logging.getLogger("neuromorphic_mcp_server")

# Initialize FastMCP Server Instance
mcp = FastMCP(
    name="SpikingAttentionEngine",
    dependencies=["mlx", "mlx-snn", "numpy"]
)

# Import the local Apple Silicon optimized spiking backend
try:
    import mlx_backend as snn_engine
    backend_active = True
    logger.info("Successfully initialized local Apple Silicon spiking engine.")
except ImportError as e:
    backend_active = False
    logger.error(f"Failed to load mxsnn-based backend: {str(e)}")

@mcp.tool(
    annotations={
        "title": "Query Spiking Associative Memory",
        "readOnlyHint": True
    }
)
def query_spiking_attention(prompt_embedding: List[float], context_id: str = "default") -> str:
    """Processes a prompt vector through a persistent spiking neural network
    using STDP lateral propagation. Returns activated contextual associations
    and historical memory items to guide the LLM's next generation step.

    Args:
        prompt_embedding: The dense floating-point embedding of the current user prompt.
        context_id: A unique identifier representing the active session or task scope.
    """
    if not backend_active:
        return "Spiking attention backend is unavailable. Standard fallback routing active."

```

```

try:
    # Convert incoming list into a native MLX Array
    embedding_arr = mx.array(prompt_embedding)

    # Pass representation into the mlx-snn associative fabric
    activated_associations = snn_engine.simulate_spiking_retrieval(
        embedding=embedding_arr,
        context_id=context_id
    )

    # Format the associative output as structured prompt context
    return activated_associations
except Exception as e:
    logger.error(f"Error during spiking memory retrieval: {str(e)}")
    return f"Spiking attention error occurred: {str(e)}"

@mcp.tool()
def trigger_sleep_consolidation() -> str:
    """Triggers an offline consolidation sequence inside the spiking network.
    Runs weight decay, synaptic clustering, and memory rescaling to optimize
    the local model's parameters and free hardware resources.
    """
    if not backend_active:
        return "Backend inactive."

    try:
        status = snn_engine.run_offline_consolidation()
        return f"Consolidation complete: {status}"
    except Exception as e:
        logger.error(f"Error during consolidation execution: {str(e)}")
        return f"Consolidation failure: {str(e)}"

if __name__ == "__main__":
    # Start the server on local stdio transport
    mcp.run()

```

Apple Silicon Spiking Engine Backend (mlx_backend.py)

This module manages the execution of the neuromorphic network, performing embedding standardization, sparse spike encoding, and state updates via mlx-snn.

Python

```
import mlx.core as mx
```

```
import mxsnn
```

```
from typing import Dict, Any, List
```

```
class SpikingAttentionBackend:
```

```
    def __init__(self, dimension: int = 1024, num_neurons: int = 5000):
```

```
        self.dimension = dimension
```

```
        self.num_neurons = num_neurons
```

```
        # Configure a Leaky Integrate-and-Fire neuron layer natively in mlx-snn
```

```
        self.lif_layer = mxsnn.Leaky(beta=0.95, learn_threshold=True)
```

```
        # Initialize explicit state dictionary
```

```
        self.state = self.lif_layer.init_state(batch_size=1, features=self.num_neurons)
```

```
        # Synaptic connection weight matrix (excitatory-inhibitory balanced)
```

```
        # Using native MLX array initialization
```

```
        self.W_syn = mx.random.normal((dimension, num_neurons)) * 0.01
```

```
        self.W_lateral = mx.random.normal((num_neurons, num_neurons)) * 0.002
```

```
        # Persistent episodic memory references
```

```
        self.memory_mapping: Dict[int, str] = {}
```

```
        self.active_traces = mx.zeros((num_neurons,))
```

```
    def encode_to_spikes_top_k(self, embedding: mx.array, k: int = 150) -> mx.array:
```

```
        """Translates a dense vector embedding into a sparse population code
```

```
        using standardized z-scores.
```

```
        """
```

```
        mean = mx.mean(embedding)
```

```
        std = mx.sqrt(mx.var(embedding) + 1e-5)
```

```
        z_scores = (embedding - mean) / std
```

```
        # Retrieve value at k-th percentile using sorting/partitioning
```

```
        sorted_scores = mx.sort(z_scores)
```

```
        threshold_val = sorted_scores[-k]
```

```
        # Binary spiking mask
```

```
        spikes = mx.where(z_scores >= threshold_val, 1.0, 0.0)
```

```
return spikes
```

```
def run_snn_cycle(self, sensory_spikes: mx.array, steps: int = 20) -> mx.array:
```

```
    """Simulates the lateral propagation of spikes over a finite set of steps,  
    recording localized neural activation.
```

```
    """
```

```
    # Projects sensory spikes through synaptic projections
```

```
    input_current = mx.matmul(sensory_spikes, self.W_syn)
```

```
    # Create a local state reference
```

```
    current_state = self.state
```

```
    accumulated_spikes = mx.zeros((self.num_neurons,))
```

```
    for t in range(steps):
```

```
        # Integrate current and lateral spiking feedback
```

```
        lateral_current = mx.matmul(accumulated_spikes, self.W_lateral)
```

```
        total_current = input_current + lateral_current
```

```
        # Forward step through mx-snn LIF module
```

```
        spk, current_state = self.lif_layer(total_current, current_state)
```

```
        # Accumulate output spikes functionally
```

```
        accumulated_spikes = accumulated_spikes + spk
```

```
    # Update explicit simulation state
```

```
    self.state = current_state
```

```
    return accumulated_spikes
```

```
# Singleton instantiation for localized process scope
```

```
_engine_instance = SpikingAttentionBackend()
```

```
def simulate_spiking_retrieval(embedding: mx.array, context_id: str) -> str:
```

```
    """Simulates top-down attention and lateral propagation to select context.
```

```
    """
```

```
    sensory_spikes = _engine_instance.encode_to_spikes_top_k(embedding)
```

```
    firing_signature = _engine_instance.run_snn_cycle(sensory_spikes)
```

```
    # Retrieve indices of maximum activity
```

```
    active_neurons = mx.argsort(firing_signature)[-5:]
```

```
    active_indices = active_neurons.tolist()
```

```
    # Map firing indices to structured context segments
```

```
    retrieved_concepts =
```

```
    for idx in active_indices:
```

```

        if idx in _engine_instance.memory_mapping:
            retrieved_concepts.append(_engine_instance.memory_mapping[idx])

    if not retrieved_concepts:
        return "No high-salience spiking patterns registered. Fallback context active."

    return " / ".join(retrieved_concepts)

def run_offline_consolidation() -> str:
    """Simulates synaptic homeostasis, decaying baseline connections and consolidated pathways.
    """
    # Exponent-based decay mimicking biological synaptic normalization
    _engine_instance.W_lateral = _engine_instance.W_lateral * 0.98
    _engine_instance.W_syn = _engine_instance.W_syn * 0.99

    # Flush current transient potentials to prepare for subsequent execution cycles
    _engine_instance.state["mem"] = mx.zeros_like(_engine_instance.state["mem"])
    return "Homeostasis decay and synaptic scaling complete."

```

Codex Configuration (~/.codex/config.toml)

This configuration registers the stdio process within the Codex runtime workspace.

```

[ini, TOML]
# General Model Context Protocol Client Settings for local execution
mcp_oauth_callback_port = 5555

[mcp_servers.spiking_attention]
command = "uv"
args = [
    "--directory"
    "/Users/localuser/projects/neuromorphic-mcp"
    "run"
    "mcp_server.py"
]
startup_timeout_sec = 15
tool_timeout_sec = 30
default_tools_approval_mode = "auto"
enabled = true

```

```
[mcp_servers.spiking_attention.env]  
PYTHONPATH  "/Users/localuser/projects/neuromorphic-mcp"  
MLX_DEVICE  "gpu"
```

Claude Desktop Configuration (~/.Library/Application Support/Claude/claude_desktop_config.json)

```
JSON  
{  
  "mcpServers":  
    "spiking_attention":  
      "command": "uv"  
      "args":  
        "--directory"  
        "/Users/localuser/projects/neuromorphic-mcp"  
      "run":  
        "mcp_server.py"  
      "env":  
        "PYTHONPATH" "/Users/localuser/projects/neuromorphic-mcp"  
        "MLX_DEVICE"  "gpu"  
    }  
}
```

Claude Code Project-Scoped Configuration (.mcp.json)

```
JSON  
{  
  "mcpServers": {  
    "spiking_attention": {
```

```

    "type": "stdio",
    "command": "uv",
    "args": [
        "run",
        "mcp_server.py"
    ],
    "env": {
        "PYTHONPATH": "${CLAUDE_PROJECT_DIR}",
        "MLX_DEVICE": "gpu"
    },
    "timeout": 30000
  }
}
}
```

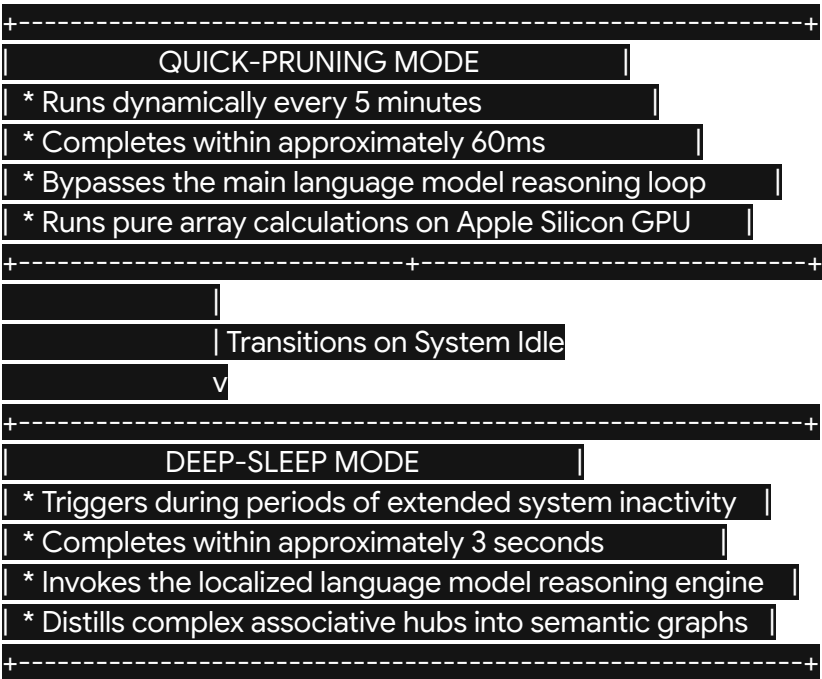
Consolidation Pipelines and Memory Pruning Lifecycle

Consolidating and pruning neural connections prevents memory overload and manages resource usage in local systems.¹⁹ Similar to biological consolidation during sleep states, an async task runner runs periodically to execute synaptic pruning.⁸ This task cleans up weak, isolated pathways while reinforcing highly correlated, structured connection hubs.¹⁴ The offline processing architecture uses a structured sequence to manage this consolidation:

Phase	System Process	Core Mathematical Operation	Downstream Cognitive Function
Phase 1	Connection Weight Decay	$W_{ij} \leftarrow W_{ij} \cdot$ ³⁴	Lowers weight values for weak connections ³⁴
Phase 2	Synaptic Clustering	Density-based connection profiling ³⁴	Identifies overlapping spiking patterns ³⁴
Phase 3	Semantic Merging	Mathematical node pooling ³⁴	Consolidates redundant memory paths ³⁴
Phase 4	Threshold Rescoring	Adaptive adjustments to V_{thr}	Keeps firing rates in healthy, balanced

		34	ranges
Phase 5	Trace Promotion	Long-term Synaptic Facilitation ³⁵	Moves active traces to persistent storage ³⁴
Phase 6	Relationship Extraction	Hebbian Distillation ¹⁴	Builds structured semantic connection graphs ¹⁴
Phase 7	Neurogenesis	State re-initialization ³⁴	Frees up inactive nodes for new memory traces ³⁴

To minimize execution overhead on consumer hardware, this consolidation process is divided into two distinct runtime modes³⁴:



During conversational gaps, the quick-pruning process runs directly on the local GPU,

preserving system resources and preventing thermal spikes.⁵ When the host enters an idle state (e.g., during developer breaks), the deep-sleep routine consolidates transient traces into a structured, long-term memory graph.⁸ This dual-mode approach keeps the local spiking network efficient and prevents memory overhead during active development.¹⁴

Nuanced Architectural Conclusions and Future Directives

The implementation of a localized neuromorphic attention spiking plugin represents a significant shift in how memory and context are managed in consumer hardware. By using on-chip spiking networks rather than traditional vector databases, the architecture changes how context is represented: it is treated as a dynamic, evolving neural substrate shaped directly by experience, rather than a collection of static document coordinates.⁸

Integrating this system via the Model Context Protocol (MCP) using the `mlx-snn` library on Apple Silicon delivers a highly efficient, production-ready local stack with several clear advantages:

- **Resource and Power Preservation:** Replacing dense matrix calculations with sparse, addition-only integrate-and-fire dynamics reduces memory bandwidth requirements.¹ This minimizes thermal output and preserves battery life on portable Mac platforms.¹
- **Elimination of the Memory Wall:** Apple's unified memory architecture allows the system to process large temporal windows and maintain expansive spiking networks without the high cost of transferring data between the CPU and GPU.⁵
- **True Context-Aware Recall:** The spiking associative network mimics biological prefrontal cortex mechanisms.¹⁹ It automatically surfaces relevant historical relationships based on temporal patterns and lateral propagation, bypassing the limitations of simple keyword similarity search.⁸

To ensure a successful deployment, developers should first verify their configuration files using the built-in MCP diagnostic tools:

```
Bash
# Launch the interactive local MCP Inspector to verify transport channels
npx @anthropic-ai/mcp-inspector uv run mcp_server.py
```

This verifies that the local tools, state variables, and resource schemas are fully validated before linking them to production surfaces like Codex or Claude.³³ Adopting this localized, neuromorphic approach prepares the local AI stack for emerging real-time cognitive tasks, delivering an efficient, secure, and responsive offline companion.

Works cited

1. Attention via Synaptic Plasticity is All You Need: A Biologically Inspired Spiking Neuromorphic Transformer - arXiv, accessed June 25, 2026, <https://arxiv.org/html/2511.14691v1>
2. [2511.14691] Attention via Synaptic Plasticity is All You Need: A Biologically Inspired Spiking Neuromorphic Transformer - arXiv, accessed June 25, 2026, <https://arxiv.org/abs/2511.14691>
3. [2410.08711] On-Chip Learning via Transformer In-Context Learning - arXiv, accessed June 25, 2026, <https://arxiv.org/abs/2410.08711>
4. A spiking neural network inspired by neuroscience and psychology for Western mode- and key-conditioned music learning and composition - PMC, accessed June 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC13096507/>
5. mlx-snn: Spiking Neural Networks on Apple Silicon via MLX - arXiv, accessed June 25, 2026, <https://arxiv.org/html/2603.03529v1>
6. mlx-snn: Spiking Neural Networks on Apple Silicon via MLX - arXiv, accessed June 25, 2026, <https://arxiv.org/pdf/2603.03529>
7. Sorbet: A Neuromorphic Hardware-Compatible Transformer-Based Spiking Language Model - arXiv, accessed June 25, 2026, <https://arxiv.org/html/2409.15298>
8. EMBER: Autonomous Cognitive Behaviour from Learned Spiking Neural Network Dynamics in a Hybrid LLM Architecture - arXiv, accessed June 25, 2026, <https://arxiv.org/html/2604.12167v1>
9. [2604.12167] EMBER: Autonomous Cognitive Behaviour from Learned Spiking Neural Network Dynamics in a Hybrid LLM Architecture - arXiv, accessed June 25, 2026, <https://arxiv.org/abs/2604.12167>
10. EMBER: Autonomous Cognitive Behaviour from Learned Spiking Neural Network Dynamics in a Hybrid LLM Architecture : r/ResearchML - Reddit, accessed June 25, 2026, https://www.reddit.com/r/ResearchML/comments/1soy1p0/ember_autonomous_cognitive_behaviour_from_learned/
11. D-ST-Sword/mlx-snn: Spiking Neural Network library built natively on Apple MLX - GitHub, accessed June 25, 2026, <https://github.com/D-ST-Sword/mlx-snn>
12. Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU, accessed June 25, 2026, <https://machinelearning.apple.com/research/exploring-llms-mlx-m5>
13. raullenchai/awesome-mlx: A curated list of awesome projects, tools, and resources for Apple MLX — the ML framework for Apple Silicon - GitHub, accessed June 25, 2026, <https://github.com/raullenchai/awesome-mlx>
14. [2604.16839] HeLa-Mem: Hebbian Learning and Associative Memory for LLM Agents - arXiv, accessed June 25, 2026, <https://arxiv.org/abs/2604.16839>
15. What is Model Context Protocol (MCP)? A guide | Google Cloud, accessed June 25, 2026, <https://cloud.google.com/discover/what-is-model-context-protocol>
16. [Literature Review] Associative memory inspires improvements for in-context learning using a novel attention residual stream architecture - Moonlight,

accessed June 25, 2026,

<https://www.themoonlight.io/en/review/associative-memory-inspires-improvements-for-in-context-learning-using-a-novel-attention-residual-stream-architecture>

17. A BIOLOGICALLY PLAUSIBLE ASSOCIATIVE MEMORY NETWORK - OpenReview, accessed June 25, 2026, <https://openreview.net/forum?id=u4YzOzEMfR>
18. EM-LLM: Human-inspired Episodic Memory for Infinite Context LLMs, accessed June 25, 2026, <https://em-llm.github.io/>
19. Context Gating in Spiking Neural Networks: Achieving Lifelong Learning through Integration of Local and Global Plasticity - arXiv, accessed June 25, 2026, <https://arxiv.org/html/2406.01883v1>
20. Context-modular memory networks support high-capacity, flexible, and robust associative memories | bioRxiv, accessed June 25, 2026, <https://www.biorxiv.org/content/10.1101/2020.01.08.898528v1.full>
21. Creating a Model Context Protocol Server: A Step-by-Step Guide, accessed June 25, 2026, <https://michaelwapp.medium.com/creating-a-model-context-protocol-server-a-step-by-step-guide-4c853fbf5ff2>
22. Codex with Azure OpenAI in Microsoft Foundry Models, accessed June 25, 2026, <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/codex>
23. Claude Code MCP Servers: How to Connect, Configure, and Use Them - Builder.io, accessed June 25, 2026, <https://www.builder.io/blog/claude-code-mcp-servers>
24. MCP Authentication in Claude Code 2026 Guide - Truefoundry, accessed June 25, 2026, <https://www.truefoundry.com/blog/mcp-authentication-in-claude-code>
25. Build an MCP server - Model Context Protocol, accessed June 25, 2026, <https://modelcontextprotocol.io/docs/develop/build-server>
26. Model Context Protocol - Codex | OpenAI Developers, accessed June 25, 2026, <https://developers.openai.com/codex/mcp>
27. Introducing Codex Plugin for Claude Code - OpenAI Developer Community, accessed June 25, 2026, <https://community.openai.com/t/introducing-codex-plugin-for-claude-code/1378186>
28. Claude Code settings - Claude Code Docs, accessed June 25, 2026, <https://code.claude.com/docs/en/settings>
29. Connect Claude Code to tools via MCP, accessed June 25, 2026, <https://code.claude.com/docs/en/mcp>
30. Authentication - Codex - OpenAI Developers, accessed June 25, 2026, <https://developers.openai.com/codex/auth>
31. Building an MCP Server with FastAPI and FastMCP - Speakeasy, accessed June 25, 2026, <https://www.speakeasy.com/mcp/framework-guides/building-fastapi-server>
32. CLI - FastMCP, accessed June 25, 2026, <https://gofastmcp.com/cli/overview>
33. Connect to MCP servers - Claude Code Docs, accessed June 25, 2026, <https://code.claude.com/docs/en/mcp-quickstart>

34. I built a 5-tier biologically-inspired memory architecture for AI at 18 - Reddit, accessed June 25, 2026,
https://www.reddit.com/r/ArtificialIntelligence/comments/1u563ub/i_built_a_5tier_biologicallyinspired_memory/
35. Simulations of working memory spiking networks driven by short-term plasticity - Frontiers, accessed June 25, 2026,
<https://www.frontiersin.org/journals/integrative-neuroscience/articles/10.3389/fnint.2022.972055/full>
36. Add and manage MCP servers in VS Code, accessed June 25, 2026,
<https://code.visualstudio.com/docs/agent-customization/mcp-servers>